

Blogify → Autonomous Content Operations Platform

Implementation roadmap and technical planner for evolving Blogify from a CRUD blogging app into a multi-agent, self-improving content system.

Why this matters

A blogging CRUD app with an AI-wrapper endpoint reads as "I called an API." This roadmap converts the same codebase into a system that plans, uses tools, critiques its own output, learns from performance data, and only asks a human to approve the final step. That is the difference between a project and a platform — and it is what turns an interview from "tell me about your blog" into "walk me through your agent architecture."

Guiding principles

- Agentic, not scripted — agents decide which tools to call and when, not a fixed if/else chain.
- Every agent action is logged and inspectable — observability is a feature, not an afterthought.
- Human-in-the-loop before anything goes live — autonomy with a safety valve, the way production systems are actually built.
- Build on what already exists — reuse the encryption, OAuth, Cloudinary, and rate-limiting patterns already in the codebase rather than bolting on parallel systems.
- Each phase should be independently demoable — so the project has value even if later phases run out of time.

Target end-state architecture

Goal in → Planner agent → Research agent (tool use) → Writer agent → Critic agent (revise loop) → Human approval → Publish/distribute agent → Analytics agent → feedback into Planner. A parallel RAG layer lets readers query the blog's own content directly.

PHASE 0 Foundation — agent infrastructure

Before any "agent" can exist, you need the scaffolding: a place to store agent state, a way to log every decision and tool call, and real function-calling instead of a single prompt-response. Skipping this phase is why most "AI features" end up being wrappers.

Feature	What to build	Key tech
AgentRun model	New Mongoose model tracking a content job end-to-end: status enum (planning → researching → drafting → critiquing → revising → awaiting_approval → publishing → done/failed), goal, current step, retry count, linked blogId once published.	MongoDB / Mongoose
AgentStep log	Sub-collection or embedded array on AgentRun storing each step: agent name, input, output, tool calls made, tokens used, timestamp, and reasoning summary. This is what powers the trace viewer in Phase 1.	MongoDB
Tool-calling wrapper	Replace single-shot prompt calls in llm.js with real function-calling: define JSON-schema tools (search_web, check_existing_posts, get_analytics) and let the model choose which to invoke, looping until it returns a final answer.	LiteLLM + Groq function calling
Job queue	Agent runs should execute asynchronously and be resumable/retryable, not block a request thread. A simple queue keeps this production-realistic.	BullMQ + Redis, or node-cron for MVP
Agent run API + trace UI	REST endpoints (GET /agent-runs, GET /agent-runs/:id) and a new EJS view rendering the full step-by-step trace — this becomes the single most impressive screen in a demo.	Express routes + EJS

Resume line

"Designed an agent execution framework with persisted state machines, structured tool-calling, and a full decision trace viewer — the same pattern used in production agent observability tools like LangSmith."

PHASE 1 Multi-agent content pipeline

This is the centerpiece: replace the single /ai/draft endpoint with a chain of specialized agents that plan, research, write, critique, and revise — with a human approval checkpoint before anything publishes.

Feature	What to build	Key tech
Planner agent	Takes a high-level goal ("write about X" or "grow traffic in topic Y") and produces a task plan: what to research, target keywords, structure outline, which tools to use.	LLM + tool-calling
Research agent	Executes the plan's research steps: calls <code>search_web</code> for current information, calls <code>check_existing_posts</code> to avoid duplicating the user's own content, summarizes findings with sources.	Web search tool, vector similarity
Writer agent	Drafts the full post from the plan + research, in the site's existing <code>DRAFT_PROMPT</code> style, but now grounded in retrieved sources instead of the model's own memory.	LiteLLM / Groq
Critic agent	Independently scores the draft on accuracy vs. sources, tone, structure, and SEO basics. Returns approve or reject-with-reasons. Rejections route back to the writer agent (max N retries) — this loop is the core "agentic" behavior to highlight in interviews.	LLM-as-judge pattern
Human approval UI	Dashboard showing pending drafts with the full trace (plan, sources, critique history) and Approve / Edit / Reject actions before anything reaches the Blog collection.	EJS + existing auth middleware
Publish/distribute agent	On approval, creates the Blog document (reusing existing slug logic), then fans out to configured channels — reusing the existing <code>tweetNewPost.js</code> pattern, generalized into a provider-adapter interface.	twitter-api-v2, provider adapters

Resume line

"Built a multi-agent content pipeline with planner, researcher, writer, and critic agents collaborating through a self-correcting revise loop, gated by a human approval step before publish."

PHASE 2 RAG-powered "ask my blog" chatbot

A reader-facing feature that is also a legitimate demonstration of retrieval-augmented generation — one of the most interview-relevant skills you can put on a resume right now.

Feature	What to build	Key tech
Chunking pipeline	On publish (or as a backfill script), split each blog body into overlapping chunks (e.g. ~500 tokens, 50-token overlap), preserving heading context per chunk.	Custom chunker or langchain text splitters
Embeddings + vector store	Embed each chunk and store the vector alongside blogId, chunk text, and position. Use MongoDB Atlas Vector Search if already on Atlas, or pgvector/a lightweight local store otherwise.	OpenAI/Groq embeddings, Atlas Vector Search or pgvector
Retrieval endpoint	Given a user question, embed the query, retrieve top-k relevant chunks across all posts, and pass them as grounding context to the LLM.	Cosine similarity search
Answer + citations	Generate an answer strictly from retrieved chunks, with inline links back to the source post(s) — this citation behavior is what makes it "RAG" rather than "chat."	LiteLLM / Groq
Chat widget	A small floating chat UI on the blog's public pages, reusing the site's existing Bootstrap styling.	EJS + vanilla JS or a small React island

Resume line

"Implemented a full RAG pipeline — chunking, embedding, vector retrieval, and grounded generation with citations — letting readers query the entire blog corpus conversationally."

PHASE 3 Analytics agent & self-improving loop

What makes the system "autonomous" rather than "scheduled" is that it learns from outcomes. This phase closes the loop from Phase 1's publish step back into the planner.

Feature	What to build	Key tech
View/engagement tracking	Lightweight tracking of page views, read time, and comment counts per post — the raw signal the analytics agent reasons over.	Simple middleware + MongoDB counters
Analytics agent	Runs on a schedule (e.g. weekly), reviews recent post performance vs. the content calendar's traffic predictions, and identifies underperforming or high-potential topics.	node-cron + LLM reasoning over aggregated stats
Meta-critic / prompt-improvement loop	Aggregates the critic agent's past rejection reasons (from AgentStep logs in Phase 0) and periodically proposes adjustments to prompts or the research strategy — a genuinely novel "agent improving agents" feature.	LLM reasoning over historical AgentStep data
Feedback into planner	Analytics agent output becomes input to the next planning cycle, so the system autonomously decides what to write about next based on real results, not just a static content calendar.	Existing planner agent, extended

Resume line

"Closed the feedback loop with an analytics agent that reviews content performance and autonomously adjusts future planning — and a meta-critic that improves the critique process itself based on historical rejection patterns."

PHASE 4 Distribution, security & polish

Rounds out the platform and hardens what is now a system that acts autonomously and holds more sensitive data (analytics, embeddings, agent traces).

Feature	What to build	Key tech
Additional publish channels	Generalize the provider-adapter interface from Phase 1 to add LinkedIn or Mastodon, proving the abstraction actually generalizes.	Provider adapter pattern
Scheduled / calendar-driven runs	Let a user approve a content calendar once, then have the pipeline auto-trigger AgentRuns for each planned topic on schedule.	node-cron
Rate limiting + CSRF	Extend the existing AI rate limiter to cover login/signup; add CSRF protection given cookie-based JWT auth.	express-rate-limit, csrf
Refresh-token rotation	Add refresh/rotation to the app's own JWT auth, mirroring the pattern already used for Twitter OAuth tokens.	JWT refresh tokens
Eval harness	A small test suite that runs fixed inputs through the critic agent and checks score consistency — demonstrates you think about evaluating LLM systems, not just building them.	Jest / Vitest + fixture prompts

Suggested build order & effort

Ordered for maximum demoable progress at every stage — if you stop after any phase, you still have something interview-ready.

Phase	Focus	Rough effort	Demo payoff
0	Agent infra, logging, tool-calling, trace viewer	1–2 weeks	"Here's how I built agent observability"
1	Planner/researcher/writer/critic pipeline + approvals	2–3 weeks	"Watch the agents plan, research, and self-correct live"
2	RAG chatbot over blog content	1–2 weeks	"Ask my blog anything, with citations"
3	Analytics agent + feedback loop	1–2 weeks	"It gets better on its own over time"
4	Distribution, security hardening, evals	1–2 weeks	"It's production-grade, not a demo"

Minimum viable "wow" if time is short

If only one phase is feasible before an interview, build Phase 0 + a trimmed Phase 1 (planner → writer → critic, skip research tool-use) with the trace viewer. The trace viewer is what visually proves "agent," not the polish of the output.

Key files/modules touched (mapped to existing codebase)

- **/models** — add AgentRun, AgentStep, PostAnalytics, EmbeddingChunk
- **/services** — add planner.js, researcher.js, writer.js, critic.js, analyticsAgent.js, ragRetrieval.js; extend llm.js with tool-calling; generalize tweetNewPost.js into publishers/*.js
- **/routes** — add agent.js (trigger + trace endpoints), chat.js (RAG endpoint)
- **/middlewares** — extend rateLimitAI.js patterns to cover new agent endpoints
- **/views** — add agent-run.ejs (trace viewer), approvals.ejs (human checkpoint), chat-widget partial